Hand-in Exercise 3

Deadline: April 13, 13:15

Read these instructions carefully before you start coding.

This third and final hand-in exercise covers material from the **lectures 7 through 9**. Unless noted otherwise, you are expected to code up your own routines using the algorithms discussed in class and **cannot use special library functions** (except exp(), log(), etc.). Things like `numpy` arrays and extremely simple functions like `arange`, `linspace` or `hist` (without using advanced features) are OK. Things like matrix classes, factorials and gamma functions are not. Any type of algorithm discussed in class should of course always be coded by yourself (e.g. never use `np.interp` or `scipy.interpolate`). When in doubt, write your own, or ask us on the discussion boards.

You can use the routines you wrote for the tutorials, however, your routines must be written **by yourself** (and so **in no part** by someone else or an LLM). Codes will be checked for blatant copying (with other students as well as other sources), routines which are too similar get **zero points** at best.

Start with the template provided through this link. For every main question you should write a **separate program**/separate code file(s). We **must** be able to run everything with **a single call to a script** `run.sh`, which downloads any data (data needed for an exercise **may not** be included in your code package but needs to be retrieved at runtime, see the template's run.sh), runs your scripts and generates a PDF containing all your source code and outputs **in the following format**:

- Per main question, the code of any shared modules.
- Per sub-question, an explanation of what you did.
- Per sub-question, the code specific to it.
- Per sub-question, the output(s) along with discussion/captions.

Your code may be **handed in** however you'd like, for example by emailing a zip file or by sharing a github repository. If you organize your code in multiple folders, `run.sh` should be **in the top folder**. It's a good idea to have a fellow student **test** that your code works by running ./run.sh to make sure there are no permission errors or the like. Exercises that are not run with this single command or do not have their code and output in the PDF get **zero points** (this includes solutions in Jupyter notebooks).

Ensure (**test!**) that your code runs to completion on a Virtual Desktop machine (normal workstation)[1], using the **default** version and packages of `python3` (e.g. don't load modules only installed for you). Your code should have a total run time of **at most 10 minutes**, solutions generated will not be checked beyond this limit. All scripts should run to the end without user interference, so do not ask for input and **do not use plt.show()**, as it stops automatic execution.

For all routines you write, **explain** how they work in the comments of your code and **argue** your choices! Similarly, whenever your code outputs something **clearly indicate** next to the output/in the PDF what is being printed. This includes **discussing your plots in their captions**. If you used any sources outside of this course for writing your code, **give these sources explicitly**.

If a part of your code **does not run**, explain what you did so far and what the problem was and still include that part of the code in the PDF for possible partial credit. If you are unable to get a routine to work but you need it for a follow-up question, use a library routine in the follow-up (and clearly indicate that and why you do so).

Writing code that is very time- or memory-inefficient may incur a small **penalty** (e.g. using two matrices for an LU decomposition). Additional investigations/relevant plots may earn a small **bonus** (e.g. consistency/convergence checks).

Each sub-question starts with a reference to a relevant tutorial (T) and/or a reference to the relevant lecture (L). For example, *[L2,T2.3]* means the question is related to lecture 2 and question 3 in tutorial 2. Your previously written code for these will be a great starting point!

Finally, after handing in your code, **plan a short conversation** (5 min) with the lecturer to explain your choices by signing up for a slot though this link.

[1] `https://vdesk.strw.leidenuniv.nl`; for more info, see here (may only be accessible from STRW computers).

1. **Satellite galaxies around a massive central – part 2**
Recall the number density satellite profile from the second assignment:

$$n(x) = A\,\langle N_{\rm sat}\rangle \left(\frac{x}{b}\right)^{a-3} \exp\left[-\left(\frac{x}{b}\right)^{c}\right] \tag{1}$$

Here x is the radius relative to the virial radius, $x \equiv r/r_{\rm vir}$, with $x < x_{\rm max} = 5$, and a, b and c are free parameters. $A = A(a,b,c)$ normalizes the profile and $\langle N_{\rm sat}\rangle$ is the mean number of satellites in each halo.

(a) (6 points) *[L7,T7.1]* The number of satellites in the infinitesimal range $[x, x+\mathrm{d}x)$ is given by $N(x)\mathrm{d}x = n(x)4\pi x^2\mathrm{d}x$. Find the maximum of $N(x)$ for $x \in [0,5)$ using a maximization algorithm of your choice. Output both x and $N(x)$ at the maximum. Assume $a = 2.4$, $b = 0.25$, $c = 1.6$, $x_{\rm max} = 5$, $\langle N_{\rm sat}\rangle = 100$ and $A = 256/(5\pi^{3/2})$.

In what follows, we're going to fit the model to some data. Have your `run.sh` download these files (do NOT directly include them in the solutions you hand in, they are too large):

```
https://home.strw.leidenuniv.nl/~daalen/Handin_files/satgals_m11.txt
https://home.strw.leidenuniv.nl/~daalen/Handin_files/satgals_m12.txt
https://home.strw.leidenuniv.nl/~daalen/Handin_files/satgals_m13.txt
https://home.strw.leidenuniv.nl/~daalen/Handin_files/satgals_m14.txt
https://home.strw.leidenuniv.nl/~daalen/Handin_files/satgals_m15.txt
```

Each file contains haloes in a certain halo mass bin with variable numbers of satellites. For example, the first file contains over 20 000 000 low-mass haloes, most of which contain zero satellites, and the last contains only two (cluster-sized) haloes, each containing several hundred satellites. Reading and plotting code is included in the template.

Since satellite galaxies are discrete objects, we are dealing with a Poisson realization here, and we should therefore write down a Poisson likelihood. In the following exercises, we will compare the results of an unbiased fit with that of an "easy" χ^2 fit with the correct Poisson variance $\sigma^2 = \mu$. Regardless of how we fit the profile, keep in mind that A depends on the parameters a, b and c, so recalculate A with your own integration routine (assignment 2) at every step!

(b) (8 points *[+2]*) *[L8,T8.2]* First we'll take the (incorrect but easy) χ^2 approach:

1. Choose any number of radial bins, space (real or log), and range (e.g. $x \in [0, x_{\rm max}]$). Argue your choices.
2. Then, separately for each file, calculate $\langle N_{\rm sat}\rangle$ and bin the satellite galaxy radii to get a mean number of satellites *per halo* in each radial bin, N_i.
3. Minimize a χ^2 for each, using that both the model mean and variance for bin $[x_i, x_{i+1}]$ are given by $\tilde{N}_i = 4\pi \int_{x_i}^{x_{i+1}} n(x)x^2\,\mathrm{d}x$. You can reuse the minimization routine you wrote for part (a), or, for 2 bonus points, write and use a derivative-based minimization routine (but only if the derivatives are correct).[2]
4. For each of the five data sets, report $\langle N_{\rm sat}\rangle$, your best-fit a, b and c, and the minimum value for χ^2. Also plot your binned data together with the best-fit profiles in a log-log plot ($\tilde{N}_i$ vs x).

(c) (5 points) *[L8,T8.2]* Now we take the correct, unbiased approach. Write down a Poisson log-likelihood for model counts $\mu = \tilde{N}_i$ and N_i mean observed counts in each bin. Explain the steps you take. Either use the same bins as in (b) or no bins at all. For each of the five data sets, minimize $-\ln\mathcal{L}(a,b,c)$ using the minimization routine you wrote for part (a) or part (b) (or write a new one), and again report your best-fit a, b and c, and the minimum value for $-\ln\mathcal{L}(a,b,c)$. Also plot your binned data together with the best-fit profiles in a log-log plot.

(d) (5 points *[+2]*) *[L9]* Now we want to see which of the two methods yielded the models most consistent with the data. Since the data is binned, we'll do a G-test (if you did not use bins for (c), bin the best-fit model by integrating over the bins from (b), or, for 2 bonus points, use

[2] If you choose the downhill simplex method (which does not earn bonus points), make sure you start with a non-degenerate simplex!

a K-S test for (c) instead). Calculate the statistic G for each method and each data file (so 10 total). Think hard about what to use as input to the G-test. Argue what the number of degrees of freedom is for each data set and calculate the significance Q of G using the χ^2 distribution.[3] Comparing the Q-values for each data set between the two methods, what can you conclude?

(e) **BONUS QUESTION** *(4 points) [L9]* For bonus points, do a Monte Carlo simulation, as follows. Choose one particular data file and take your corresponding best-fit parameters for both (b) and (c). For both sets of parameters, generate another set of haloes, using the same number of haloes and $\langle N_{\rm sat} \rangle$ as in the original data files, by sampling from these distributions as in assignment 2. Find the best-fit parameters for each mock data set using the corresponding method. Repeat this process as many times as you can (using your original best-fit parameters to sample each time) and plot the best-fit profile for each sample in one plot for each method (so one plot for χ^2, one plot for Poisson likelihood). Overplot the original best fits in a different colour, and also plot the mean profile of all resamplings. Does the mean reasonably match the original profile in both cases? What can you conclude from this experiment?

[3] You can use a library function for the (incomplete) gamma functions.